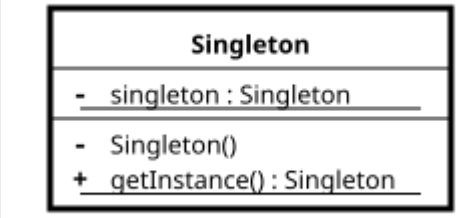
Singleton pattern

In object-oriented programming, the **singleton pattern** is a software design pattern that restricts the instantiation of a class to a singular instance. It is one of the well-known "Gang of Four" design patterns, which describe how to solve recurring problems in object-oriented software.[1] The pattern is useful when exactly one object is needed to coordinate actions across a system.



A class diagram exemplifying the singleton pattern.

More specifically, the singleton pattern allows classes to:[2]

- Ensure they only have one instance
- Provide easy access to that instance
- Control their instantiation (for example, hiding the constructors of a class)

The term comes from the mathematical concept of a singleton.

Common uses

Singletons are often preferred to global variables because they do not pollute the global namespace (or their containing namespace). Additionally, they permit lazy allocation and initialization, whereas global variables in many languages will always consume resources.[1][3]

The singleton pattern can also be used as a basis for other design patterns, such as the abstract factory, factory method, builder and prototype patterns. Facade objects are also often singletons because only one facade object is required.

Logging is a common real-world use case for singletons, because all objects that wish to log messages require a uniform point of access and conceptually write to a single source.[4]

Implementations

Implementations of the singleton pattern ensure that only one instance of the singleton class ever exists and typically provide global access to that instance.

Typically, this is accomplished by:

- Declaring all constructors of the class to be private, which prevents it from being instantiated by other objects
- Providing a static method that returns a reference to the instance

The instance is usually stored as a private static variable; the instance is created when the variable is initialized, at some point before when the static method is first called.

```
import std;

class Singleton {
private:
    Singleton() = default; // no public constructor
    ~Singleton() = default; // no public destructor
    inline static Singleton* inst = nullptr; // declaration class variable
    int value;
public:
    // defines a class operation that lets clients access its unique instance.
    static Singleton& instance() {
        if (!inst) {
            inst = new Singleton();
        }
        return *inst;
    }

    Singleton(const Singleton&) = delete("Copy construction disabled");
    Singleton& operator=(const Singleton&) = delete("Copy assignment disabled");

    static void reset() {
        delete inst;
        inst = nullptr;
    }

    // existing interface goes here
    [[nodiscard]]
    int getValue() const noexcept {
        return value;
    }

    void setValue(int v) noexcept {
        value = v;
    }
};

int main() {
    Singleton::instance().setValue(42);
    std::println("value = {}", Singleton::instance().getValue()); // prints "value = 42"
    Singleton::reset();
}
```

This is an implementation of the Meyers singleton, so named after Scott Meyers, author of *More Effective C++*, due to his recommendation of this implementation.[5] The Meyers singleton has no destruct method. The program output is the same as above.

```
import std;

class Singleton {
private:
    Singleton() = default;
    ~Singleton() = default;
    int value;
public:
    static Singleton& instance() {
        static Singleton instance;
        return instance;
    }

    [[nodiscard]]
    int getValue() const noexcept {
        return value;
    }

    void setValue(int v) noexcept {
        value = v;
    }
};

int main() {
```

```
    Singleton::instance().setValue(42);
    std::println("value = {}", Singleton::instance().getValue()); // prints "value = 42"
}
```

While the Meyers singleton is mostly unique to C++ due to C++'s function-local `static` variables with lazy thread-safe initialization, it can still be emulated in other languages. For example, in Java, one can use the initialization-on-demand holder idiom:

```
public class Singleton {
    private Singleton() {}

    private static class Holder {
        static final Singleton INSTANCE = new Singleton();
    }

    public static Singleton instance() {
        return Holder.INSTANCE;
    }
}
```

Lazy initialization

A singleton implementation may use lazy initialization in which the instance is created when the static method is first invoked. In multithreaded programs, this can cause race conditions that result in the creation of multiple instances. The following Java 5+ example[6] is a thread-safe implementation, using lazy initialization with double-checked locking.

```
public class Singleton {
    private static volatile Singleton instance = null;

    private Singleton() {}

    public static Singleton instance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

In a manner similar to the Meyers singleton, one can use `System.Lazy<T>` in C# to use lazy initialization:[7]

```
using System;

public sealed class Singleton
{
    private static readonly Lazy<Singleton> _instance = new(() => new Singleton());

    public static Singleton Instance => _instance.Value;

    private Singleton() {}
}
```

Criticism

Some consider the singleton to be an anti-pattern that introduces global state into an application, often unnecessarily. This introduces a potential dependency on the singleton by other objects, requiring analysis of implementation details to determine whether a dependency actually exists.[8] This increased coupling can introduce difficulties with unit testing.[9] In turn, this places restrictions on any abstraction that uses the singleton, such as preventing concurrent use of multiple instances.[9][10][11]

Singletons also violate the single-responsibility principle because they are responsible for enforcing their own uniqueness along with performing their normal functions.[9]

Avoidance through dependency injection

Dependency injection can be used to avoid the singleton pattern. The singleton pattern enforces a structural restriction of only one instance of a class, but dependency injection focuses on how dependencies are passed around, avoiding the necessity of objects to look up their own global state.[12]

For example, in the following example, the `OrderService` is tightly coupled to `DatabaseConnection`; the database cannot be swapped and a different instance cannot be used.

```
// a concrete singleton class
class DatabaseConnection {
    private static DatabaseConnection instance;

    private DatabaseConnection() {
        // database resource initialization
    }

    public static synchronized DatabaseConnection instance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }

    public void executeQuery(String sql) {
        System.out.printf("Executing: %s%n");
    }
}

// a tightly-coupled consumer class
class OrderService {
    public void placeOrder(String orderId) {
        // hardcoded dependency lookup makes this impossible to unit test
        DatabaseConnection db = DatabaseConnection.instance();
        db.executeQuery(String.format("INSERT INTO orders VALUES ('%s')"));
    }
}
```

Dependency injection avoids asking how the database is created, or how many instances exist, and only asks for the `Database` interface in its constructor:

```
// an abstract interface
interface Database {
    void executeQuery(String sql);
}

// implement the concrete class (no singleton logic)
```

```java
class SqlDatabase implements Database {
    @Override
    public void executeQuery(String sql) {
        System.out.println("Executing SQL: %s", sql);
    }
}

// the consumer class accepts dependency via constructor injection
class OrderService {
    private final Database db;

    // The dependency is injected from the outside
    public OrderService(Database db) {
        this.db = db;
    }

    public void placeOrder(String orderId) {
        db.executeQuery(String.format("INSERT INTO orders VALUES ('%s')", orderId));
    }
}
```

See also

- Initialization-on-demand holder idiom
- Multiton pattern
- Dependency injection
- Software design pattern

References

1. Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides (1994). *Design Patterns: Elements of Reusable Object-Oriented Software* (https://archive.org/details/designpatternsel00gamm/page/127). Addison Wesley. pp. 127ff (https://archive.org/details/designpatternsel00gamm/page/127). ISBN 0-201-63361-2.
2. "The Singleton design pattern - Problem, Solution, and Applicability" (http://w3sdesign.com/?gr=c05&ugr=proble). *w3sDesign.com*. Retrieved 2017-08-16.
3. Soni, Devin (31 July 2019). "What Is a Singleton?" (https://betterprogramming.pub/what-is-a-singleton-2dc38ca08e92). *BetterProgramming*. Retrieved 28 August 2021.
4. Rainsberger, J.B. (1 July 2001). "Use your singletons wisely" (https://web.archive.org/web/20210224180356/https://www.ibm.com/developerworks/library/co-single/). IBM. Archived from the original (https://www.ibm.com/developerworks/library/co-single/) on 24 February 2021. Retrieved 28 August 2021.
5. Scott Meyers (1997). *More Effective C++*. Addison Wesley. pp. 146 ff. ISBN 0-201-63371-X.
6. Eric Freeman, Elisabeth Freeman, Kathy Sierra, and Bert Bates (October 2004). "5: One of a Kind Objects: The Singleton Pattern" (https://books.google.com/books?id=GGpXN9SMELMC&pg=PA182). *Head First Design Patterns* (First ed.). O'Reilly Media, Inc. p. 182. ISBN 978-0-596-00712-6.
7. Microsoft Learn. "Lazy<T> Class" (https://learn.microsoft.com/en-us/dotnet/api/system.lazy-1). *learn.microsoft.com*. Microsoft Learn. Retrieved 6 July 2026.
8. "Why Singletons Are Controversial" (https://web.archive.org/web/20210506162753/https://code.google.com/archive/p/google-singleton-detector/wikis/WhySingletonsAreControversial.wiki). *Google Code Archive*. Archived from the original (https://code.google.com/archive/p/google-singleton-detector/wikis/WhySingletonsAreControversial.wiki) on 6 May 2021. Retrieved 28 August 2021.

9. Button, Brian (25 May 2004). "Why Singletons are Evil" (https://web.archive.org/web/20210715184717/https://docs.microsoft.com/en-us/archive/blogs/scottdensmore/why-singletons-are-evil). *Being Scott Densmore*. Microsoft. Archived from the original (https://docs.microsoft.com/en-us/archive/blogs/scottdensmore/why-singletons-are-evil) on 15 July 2021. Retrieved 28 August 2021.
10. Steve Yegge. Singletons considered stupid (http://steve.yegge.googlepages.com/singleton-considered-stupid), September 2004
11. Hevery, Miško, "Global State and Singletons (http://googletesting.blogspot.com/2008/11/clean-code-talks-global-state-and.html)", *Clean Code Talks*, 21 November 2008.
12. Microsoft Learn (11 March 2026). "Dependency injection guidelines" (https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection/guidelines). *learn.microsoft.com*. Microsoft Learn.

External links

- Complete article "Java Singleton Pattern Explained (https://howtodoinjava.com/design-patterns/creational/singleton-design-pattern-in-java/)"
- Four different ways to implement singleton in Java "Ways to implement singleton in Java (https://web.archive.org/web/20150709155148/http://www.javaexperience.com/design-patterns-singleton-design-pattern/)"